

UBER SUPPORT AI AGENT

An AI customer-support agent for “Uber_Support”, built from the Kaggle "Customer Support on Twitter" dataset. Classifies each message, drafts a reply grounded in Uber's historical replies and help-centre articles, and decides auto-handle vs escalate with a stated reason.

THE PROBLEM

Real, noisy Twitter support conversations. Build an agent that classifies each message, drafts a reply grounded in how the brand historically resolved similar issues, and decides auto-handle vs. escalate - with a stated reason.

THE SOLUTION

A fixed 7-step pipeline — enrich → rules → retrieve → draft → check → persist → respond. The LLM reads the message; deterministic rules decide where it goes. Not an agent loop: there are no side-effect tools to choose between.

Key finding - Uber’s Twitter team is a triage desk, not a resolution channel. Only 13.6% of replies are informative; the rest deflect (“send us a DM”). This shaped every downstream design choice.

150

hand-labelled golden rows

7

fixed pipeline steps

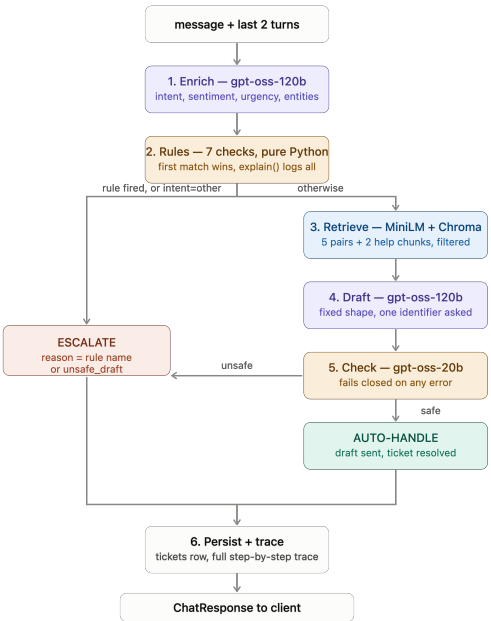
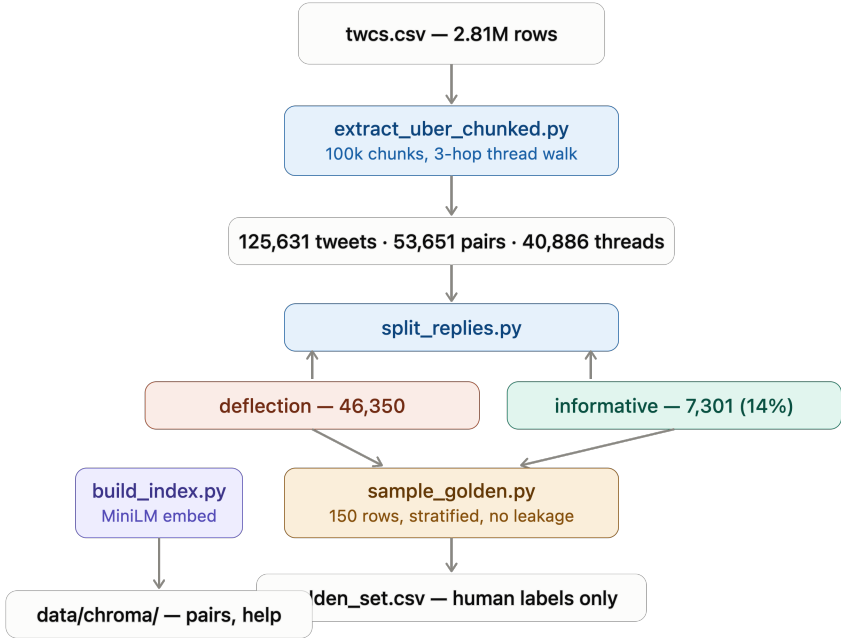
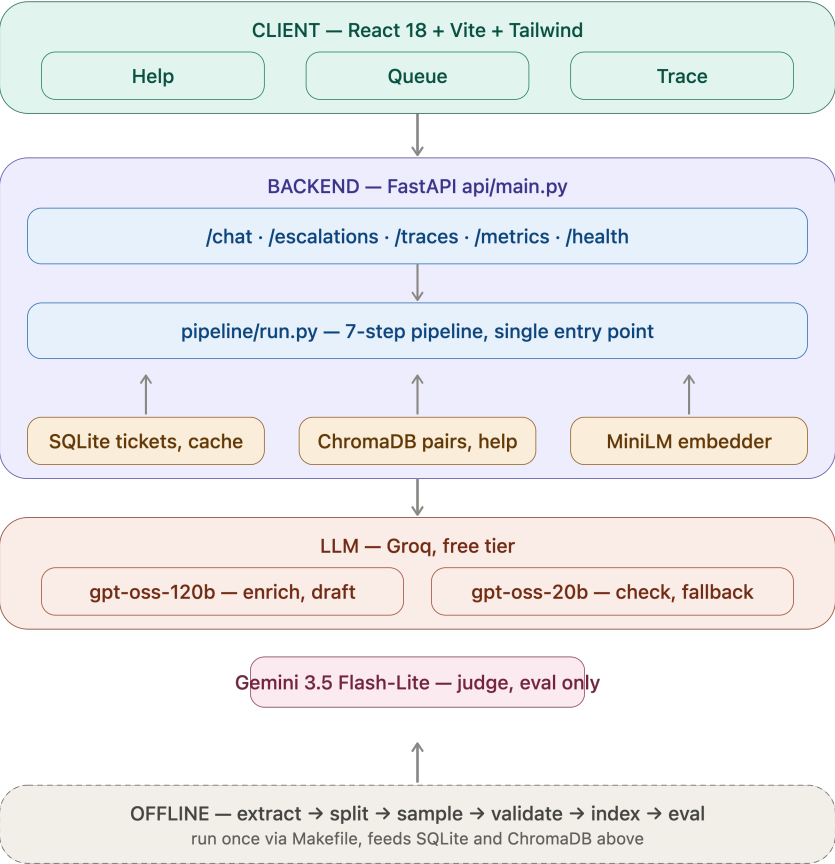
100

tests, ~2s, zero network

<15 min

reproduce, no API keys

Architecture, request flow & stack



TECH STACK

Layer	Choice
LLM	Groq gpt-oss-120b / 20b — free tier
Judge	Gemini 3.5 Flash-Lite — eval only
Embeddings	MiniLM-L6-v2, local CPU
Storage	SQLite + ChromaDB, local
API / UI	FastAPI, React + Vite + Tailwind

The rules decide; the model reads

ESCALATION RULE TABLE - first match wins, reason = rule name

#	Rule	Fires when
1	safety	mentions_safety
2	legal	mentions_legal
3	amount_over	amount > \$10
4	urgent	urgency = high
5	repeat_contact	is_repeat_contact
6	low_confidence	confidence < 0.7
7	not_allowed	intent not on allow-list

WORKED EXAMPLE — SAFETY ESCALATION

“My driver was drunk and swerving”

Enrich sets mentions_safety = true → rule 1 fires immediately → escalated with reason “safety”. The draft step never runs — the trace shows the pipeline stopping before it, not a policy the model chose to follow.

DESIGN PRINCIPLES

Fixed pipeline, not an agent loop

No side-effect tools to choose between.

Rules run before retrieval

An escalated message costs one LLM call, not three.

One enrich call

Intent + sentiment + urgency + entities, same read of the message.

Everything local & cached

Deterministic, 15-minute reproduction, zero API keys.

HANDLED MID-PROJECT

Groq deprecated the Llama models mid-build — migrated to GPT-OSS 120B/20B in two lines because model IDs are env-driven.

An early draft config returned empty replies that a checker wrongly passed as safe — fixed by rejecting empty drafts before any LLM call and never caching them.

Evaluation & key insights

HEADLINE METRICS

Metric	Value
Intent macro-F1 (ours vs. baselines)	90%
Escalation recall	90%
Judge vs. human kappa (40 rows)	89%
Labeller A vs. B kappa (50 rows)	90%

TOP FAILURE MODES — found, understood, next fix identified

- **Misdirected payouts & hacked accounts** currently auto-handle when they should escalate — next fix: a `mentions_account_compromise` rule.
- **Confidence is uncalibrated** (clusters 0.93–0.99) — the 0.7 floor may rarely fire; per-reason breakdown decides if it stays.

REPO MAP

```
pipeline/ 11 modules — models, taxonomy, enrich,
rules, retrieve, draft, check, run
eval/      run_eval.py, metrics.py, judge.py,
kappa.py
tests/     100 pytest tests, fake LLM, no
network, ~2s
docs/      PROJECT_PLAN, DECISIONS (47 entries),
TRACE_SPEC
```

WHAT'S MISLEADING ABOUT THE HEADLINE NUMBER

Small, self-labelled golden set (150 rows).
Escalation recall measured only on enumerated conditions.
Judge is an LLM too.
Uber's replies are mostly deflections, so similarity-to-Uber rewards the wrong thing.

Small, typed, explainable live

pipeline/rules.py

```
RULES = [  
    ("safety", lambda e: e.entities.mentions_safety),  
    ("legal", lambda e: e.entities.mentions_legal),  
    ("amount_over", lambda e: (e.entities.amount or 0)  
                               > AMOUNT_LIMIT),  
    ("urgent", lambda e: e.urgency == "high"),  
    ("repeat_contact", lambda e:  
                             e.entities.is_repeat_contact),  
    ("low_confidence", lambda e: e.confidence < 0.7),  
    ("not_allowed", lambda e: e.intent not in AUTO_ALLOW),  
]
```

100 tests, offline

- Fake Groq client, no network
- Every rule tested in order
- run() tested on escalate, auto, unsafe-draft, empty-draft, other
- Trace shape asserted (8 keys, 1 row per rule)
- ~2 seconds, full suite

REPRODUCE IN UNDER 15 MINUTES — NO API KEYS

```
git clone → docker compose run --rm -e CACHE_ONLY=1 api python eval/run_eval.py
```